LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



NAMA: Anindya Sukma Dwiyanda

NIM: 103132400006

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dasar teori dari praktikum ini berfokus pada pilar Pemrograman Berorientasi Objek yaitu pewarisan (*inheritance*) dan polimorfisme untuk menghindari duplikasi kode. Pewarisan memungkinkan kelas turunan (*subclass*) mewarisi atribut serta perilaku dari kelas induk (*superclass*). Dalam mekanismenya, terdapat kata kunci *super* untuk mengakses anggota kelas induk, *method overloading* untuk membuat beberapa *method* bernama sama dengan parameter berbeda, serta *method overriding* agar kelas turunan dapat menulis ulang *method* induknya.

B. Guided

1. Inheritance

```
package guided.guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

```
package guided.guided1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z = " + hasil);
    }
}
```

```
package guided.guided1;

public class Demo {
    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;
```

```
            System.out.println("Superclass: ");
            superclass.tampilXY();

            subclass.x = 1;
            subclass.y = 2;
            subclass.z = 5;

            System.out.println("Subclass: ");
            subclass.jumlahXYZ();
        }
    }
```

Output :

```
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z = 8
```

2. Keyword Super

```
package guided.guided2;

public class Employee {
    private String name;
    String departemen;

    public Employee (String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

```
package guided.guided2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
```

```
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }
}
```

```
package guided.guided2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
nama: Adi
alamat: Sukabirus
departemen: SDM
```

3. Overloading Method

```
package guided.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param){
        judul = param;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("Judul: "+ judul + ", pencipta: "+ pencipta);
    }
}
```

```
package guided.guided3;

public class Main {
```

```
        public static void main(String[] args) {
            Lagu d,e;
            d = new Lagu();
            e = new Lagu();

            d.IsiParam("Lagu 1");
            e.IsiParam("Lagu 2", "aku");

            d.tampilkan();
            e.tampilkan();
        }
    }
```

Output :

```
Judul: Lagu 1, pencipta: Tidak dikenal
Judul: Lagu 2, pencipta: aku
```

4. Overiding Method

```
package guided.guided4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik class A dipanggil...");
    }
}
```

```
package guided.guided4;

public class B extends A {
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

```
package guided.guided4;

public class Main {
    public static void main(String[] args){
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided 1

Kelas induk (model umum)
Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).
Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
Memiliki method untuk menampilkan informasi.
Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (overloading):
Tanpa parameter: biaya = jumlah data latih × 2.
Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

Jawaban :

```
package unguided;

public class ModelDasar {
    private final String identitasModel;
    private final int totalData;

    public ModelDasar(String nama, int dataTraining) {
        this.identitasModel = nama;
        this.totalData = dataTraining;
    }

    public int getTotalData() {
        return totalData;
    }

    public void cetakInfo() {
        System.out.printf("Model: %s | Data latih: %d%n", identitasModel,
totalData);
    }

    public int estimasiBiaya() {
        return totalData * 2;
    }
```

```
    public int estimasiBiaya(int jumlahEpoch) {
        return totalData * jumlahEpoch;
    }
}
```

Penjelasan :
Berfungsi sebagai superclass utama yang menyimpan atribut umum seperti nama model dan jumlah data terlatih, serta menyediakan method dasar untuk menampilkan informasi dan menghitung biaya komputasi.

2. Soal Unguided 2
   Kelas turunan pertama (model regresi)
   Mewarisi kelas induk dan tambahkan atribut jumlah fitur.
   Constructor menerima nama, jumlah data latih, dan jumlah fitur; gunakan constructor kelas induk.
   Override method: tampilkan informasi dari kelas induk.
   Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
   Tambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.

   Jawaban :

```
package unguided;

public class Regresi extends ModelDasar {
    private final int banyakFitur;

    public Regresi(String nama, int dataTraining, int fitur) {
        super(nama, dataTraining);
        this.banyakFitur = fitur;
    }

    @Override
    public void cetakInfo() {
        super.cetakInfo();
        System.out.printf("Jumlah fitur: %d%n", banyakFitur);
    }

    @Override
    public int estimasiBiaya() {
        int tambahanFitur = banyakFitur * 10;
        return super.estimasiBiaya() + tambahanFitur;
    }
```

```
    public int estimasiBiaya(int jumlahEpoch, int ukuranBatch) {
        return super.estimasiBiaya(jumlahEpoch) / ukuranBatch;
    }
}
```

Penjelasan :
Merupakan kelas turunan yang mewarisi kelas induk dengan menambahkan atribut jumlah fitur, serta melakukan *override* dan *overloading* pada method penghitung biaya

3. Soal Unguided 3
   Kelas turunan kedua (model klasifikasi)
   Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
   Constructor menerima nama, jumlah data latih, dan jumlah kelas; gunakan constructor kelas induk.
   Override method: tampilkan informasi dari kelas induk.
   Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
   Versi penghitung biaya dengan parameter epoch tidak di-override.

   Jawaban :

```
package unguided;

public class Klasifikasi extends ModelDasar {
    private final int banyakKelas;

    public Klasifikasi(String nama, int dataTraining, int kelas) {
        super(nama, dataTraining);
        this.banyakKelas = kelas;
    }

    @Override
    public void cetakInfo() {
        super.cetakInfo();
        System.out.printf("Jumlah kelas: %d%n", banyakKelas);
    }

    @Override
    public int estimasiBiaya() {
        return super.estimasiBiaya() * banyakKelas;
    }
}
```

Penjelasan :

Kelas turunan yang menambahkan atribut jumlah kelas dan menyesuaikan perhitungan biaya komputasi berdasarkan jumlah kelas tersebut.

4. Soal Unguided 4

   Kelas turunan ketiga (model klasifikasi deep learning)
   Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
   Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan constructor kelas di atasnya.
   Override method: tampilkan informasi dari kelas di atasnya.
   Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

   Jawaban:

```
package unguided;

public class DeepLearning extends Klasifikasi {
   private final int totalLayer;

   public DeepLearning(String nama, int dataTraining, int kelas, int layer) {
      super(nama, dataTraining, kelas);
      this.totalLayer = layer;
   }

   @Override
   public void cetakInfo() {
      super.cetakInfo();
      System.out.printf("Jumlah layer: %d%n", totalLayer);
   }

   @Override
   public int estimasiBiaya() {
      int biayaKlasifikasi = super.estimasiBiaya();
      int tambahanLayer = totalLayer * 200;
      return biayaKlasifikasi + tambahanLayer;
   }
}
```

   Penjelasan:
   Kelas turunan bertingkat yang mewarisi kelas klasifikasi dengan menambahkan atribut jumlah layer serta memperbarui perhitungan biaya akhir.

5. Soal Unguided 5

Class Main
Buat tiga objek berikut:
Jenis model
Nama
Data latih
Atribut tambahan
Regresi
"Regresi Linier"
500
4 fitur
Klasifikasi
"Decision Tree"
800
3 kelas
Klasifikasi deep learning
"Neural Network"
1000
5 kelas, 3 layer

Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
Setelah perulangan, tampilkan total biaya.
Terakhir, tampilkan juga:
biaya 10 epoch untuk model "Neural Network" (ambil dari array),
biaya 10 epoch untuk model "Regresi Linier",
biaya 10 epoch dengan ukuran batch 50 untuk model "Regresi Linier".

Jawaban:

```
package unguided;

public class Main {
    public static void main(String[] args) {
        Regresi regresiLinear = new Regresi("Regresi Linier", 500, 4);

        ModelDasar[] daftarUji = new ModelDasar[3];
        daftarUji[0] = regresiLinear;
        daftarUji[1] = new Klasifikasi("Decision Tree", 800, 3);
        daftarUji[2] = new DeepLearning("Neural Network", 1000, 5, 3);

        int biayaKeseluruhan = 0;
        for (int posisi = 0; posisi < daftarUji.length; posisi++) {
```

```
                ModelDasar modelAktif = daftarUji[posisi];
                modelAktif.cetakInfo();

                int biayaModel = modelAktif.estimasiBiaya();
                biayaKeseluruhan += biayaModel;
                System.out.printf("Biaya komputasi: %d%n---%n", biayaModel);
            }

            System.out.printf("Total biaya: %d%n", biayaKeseluruhan);

            int epochUji = 10;
            int ukuranKelompok = 50;
            System.out.printf("Biaya %d epoch (Neural Network): %d%n",
                    epochUji, daftarUji[2].estimasiBiaya(epochUji));
            System.out.printf("Biaya %d epoch (Regresi Linier): %d%n",
                    epochUji, regresiLinear.estimasiBiaya(epochUji));
            System.out.printf("Biaya %d epoch batch %d (Regresi Linier): %d%n",
                    epochUji, ukuranKelompok, regresiLinear.estimasiBiaya(epochUji,
        ukuranKelompok));
        }
    }
        public int estimasiBiaya(int jumlahEpoch) {
            return totalData * jumlahEpoch;
        }
}
```

Penjelasan:
Kelas eksekusi yang bertugas membuat objek dari berbagai model, menyimpannya ke dalam satu array berbasis polimorfisme, dan menampilkan hasil akhir perhitungan biaya komputasi.

Output:

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

D. Kesimpulan

Kesimpulan dari praktikum ini adalah kegiatan tersebut berhasil memberikan pemahaman mendalam mengenai penerapan hierarki kelas yang terorganisir. Penguasaan terhadap konsep pewarisan, penggunaan kata kunci *super*, serta *overloading* dan *overriding* terbukti sangat esensial dalam menyusun kode program berbasis objek yang fleksibel dan efisien.